

Program:

```
pip install numpy pandas scikit-learn gensim nltk

import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import normalize
from sklearn.decomposition import LatentDirichletAllocation
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')

# Sample documents
documents = [
    "The sky is blue and beautiful.",
    "Love this blue and beautiful sky!",
    "The quick brown fox jumps over the lazy dog.",
    "A king's breakfast has sausages, ham, bacon, eggs, toast and beans",
    "I love green eggs, ham, sausages, and bacon!",
    "The brown fox is quick and the blue dog is lazy!",
    "The sky is very blue and the sky is very beautiful today",
    "The dog is lazy but the brown fox is quick!"
]

# Preprocessing
stop_words = set(stopwords.words('english'))
def preprocess(doc):
    return ' '.join([word for word in doc.lower().split() if word not in stop_words])

processed_docs = [preprocess(doc) for doc in documents]

# LSA (Latent Semantic Analysis) using TruncatedSVD
def perform_lsa(documents, num_topics=2):
    vectorizer = TfidfVectorizer()
    X = vectorizer.fit_transform(documents)

    svd_model = TruncatedSVD(n_components=num_topics, random_state=42)
    lsa_matrix = svd_model.fit_transform(X)

    terms = vectorizer.get_feature_names_out()
```

```

for i, comp in enumerate(svd_model.components_):
    terms_in_topic = [terms[i] for i in comp.argsort()[::-6:-1]]
    print(f"Topic {i+1}: {' '.join(terms_in_topic)}")

return lsa_matrix

print("LSA Topics:")
lsa_matrix = perform_lsa(processed_docs, num_topics=2)

# Topic Modeling using LDA (Latent Dirichlet Allocation)
def perform_lda(documents, num_topics=2):
    vectorizer = CountVectorizer()
    X = vectorizer.fit_transform(documents)

    lda_model = LatentDirichletAllocation(n_components=num_topics, random_state=42)
    lda_matrix = lda_model.fit_transform(X)

    terms = vectorizer.get_feature_names_out()

    for idx, topic in enumerate(lda_model.components_):
        terms_in_topic = [terms[i] for i in topic.argsort()[::-6:-1]]
        print(f"Topic {idx+1}: {' '.join(terms_in_topic)}")

    return lda_matrix

print("\nLDA Topics:")
lda_matrix = perform_lda(processed_docs, num_topics=2)

```